

No code proof of concept: **Step 1: Define the Initial Prototype Scope**

To get the MVP (Minimum Viable Prototype) up and running, answer this first:

 Which use case are we building this prototype for first?

Choose one:

1.  **Ticket validation for an event** (e.g., concert or VIP entry)
2.  **Voter verification system** (for voting booths or digital ballots)
3.  **Access control system** (e.g., for secure building or clearance zones)
4.  **Demo-only prototype** (just to showcase the logic, no real data)

Once we pick one, we'll proceed with:

- Data inputs
- Frontend interface (if needed)
- QR/ID scanning logic
- Token logic (Mint-to-Limbo or Mint-to-Burn)
- Logging & proof trail
- Burn confirmation output

Let me know which path we start walking—and we'll build this one layer at a time. 

You said:

ticket validation and the rest falls into place do to security levels and protocols. Lets tackle the hard one first

ChatGPT said:

Update memory?

"Spencer is starting the prototype with the ticket ..."

Yes

No

Excellent—tackling ticket validation first is the smart move, Spencer. That gives us the most dynamic use case and sets the stage for all the others.

Step 2: Ticket Validation Prototype – Basic Architecture

This prototype will simulate:

- Buying a ticket → Receiving a QR code
 - Arriving at the gate → Scanning the QR
 - Verifying identity → Minting + Burning the access token
 - Logging access history
-

Minimum Components Needed

1. **Ticket Generator**

- Simulates purchase
- Outputs a QR code (randomized ID + metadata)
- Can be emailed, downloaded, or printed

2. **Scanner App (or Web Portal)**

- Used by event staff
- Scans the QR code
- Automatically pulls ticket data and initiates validation

3. Verification Layer

- Checks minimal identity input (last 4 of ID, or group ID)
- Verifies timestamp, geolocation (optional)
- Returns PASS or FAIL

4. Token Engine

- Mint-to-Burn: Token is created + immediately burned
- Logs include:
 - Ticket ID
 - Timestamp
 - Entry point
 - Hash of validation result

5. Audit Trail Dashboard

- Displays:
 - Entry records
 - Burn confirmations
 - Duplicate scan attempts

Step 3: Choose Your Build Path

We have 3 prototype options depending on tools:

Option

Tools

Outcome

No-Code (Fastest)	Glide, Zapier, n8n, Google Sheets	Clickable demo with QR gen, scanner, logic
Low-Code	Node.js + Firebase or Supabase + QR APIs	Custom MVP web app
Full Dev	React Native / Flutter + backend	Mobile app prototype (takes longer)

✅ Let's Decide:

1. Do you want a **no-code proof of concept first** (fast to build)?
2. Or do you want to sketch a custom **mobile/web app architecture** with developer specs?

We'll start building as soon as you confirm that path.

Let's get your ticket prototype live. 📄 ⚙️

You said:

no code proof of concept

ChatGPT said:

Updated saved memory

Perfect—here we go, Spencer. We'll use **no-code tools** to build a working prototype of your **ticket validation system** with **Mint-to-Burn functionality** baked in.

📄 Ticket Validation Prototype (No-Code) – Step-by-Step Plan

⚙️ Tools We'll Use:

Function	Tool
Ticket purchase & database	Google Sheets (simulates ticket registry)
QR code generation	QR Code Monkey or Google Sheets QR API

Automation logic	Zapier or n8n.io
Scanner app / interface	Glide (turns Google Sheet into mobile web app)
Logging / Burn event	Zapier to Google Sheets log , timestamped

Step 1: Create the Ticket Registry

1. **Google Sheet Columns:**

- Ticket ID
- Buyer name (optional)
- Email / phone
- QR Code (auto-generated link)
- Status: **Unused**, **Scanned**, **Burned**
- Timestamp of burn

2. We'll simulate purchase by adding entries manually or through a form.

Step 2: QR Code Generation (Auto)

- Use a script or Zap to convert each Ticket ID into a QR code URL:
 - Example: <https://yourdomain.com/validate?id=12345XYZ>
 - Embed this into Glide so scanning opens the ticket verification screen.
-

Step 3: Create the Scanner Interface

- In **Glide**:

- Connect to your Ticket Sheet
 - Use QR scanner component to read QR → lookup Ticket ID
 - Show validation result:
 -  “Access Granted – Token Burned”
 -  “Invalid or Already Used”
-

Step 4: Zapier Logic to Burn the Token

- When a QR is scanned:
 - Zapier updates the “Status” to **Burned**
 - Records the timestamp
 - (Optional) notifies the original buyer that ticket was used
-

Step 5: Logging and Audit Trail

- Use another tab in the Sheet for:
 - All burns
 - Timestamps
 - Device/location info (if available)
 - Rejected scans (duplicate attempts)
-

What You’ll End Up With:

- A working **QR-based ticketing prototype**
 - Validated at the door with a **burn-on-scan mechanism**
 - A mobile-accessible scanner that **requires no coding**
 - Full audit trail of usage
-

Ready to build it live?

I can help you:

1. Set up the exact Google Sheet structure
2. Configure Glide and Zapier step-by-step
3. Automate the QR-to-burn process

Which step should we do first? Or do you want the full template zipped and sent to get rolling?

Perfect—here we go, Spencer. We'll use **no-code tools** to build a working prototype of your **ticket validation system** with **Mint-to-Burn functionality** baked in.

Ticket Validation Prototype (No-Code) – Step-by-Step Plan

Tools We'll Use:

Function	Tool
Ticket purchase & database	Google Sheets (simulates ticket registry)
QR code generation	QR Code Monkey or Google Sheets QR API
Automation logic	Zapier or n8n.io

Scanner app / interface

Glide (turns Google Sheet into mobile web app)

Logging / Burn event

Zapier to Google Sheets log, timestamped

Step 1: Create the Ticket Registry

1. **Google Sheet Columns:**

- Ticket ID
- Buyer name (optional)
- Email / phone
- QR Code (auto-generated link)
- Status: **Unused**, **Scanned**, **Burned**
- Timestamp of burn

2. We'll simulate purchase by adding entries manually or through a form.

Step 2: QR Code Generation (Auto)

- Use a script or Zap to convert each Ticket ID into a QR code URL:
 - Example: <https://yourdomain.com/validate?id=12345XYZ>
 - Embed this into Glide so scanning opens the ticket verification screen.
-

Step 3: Create the Scanner Interface

- In **Glide**:
 - Connect to your Ticket Sheet

- Use QR scanner component to read QR → lookup Ticket ID
 - Show validation result:
 -  “Access Granted – Token Burned”
 -  “Invalid or Already Used”
-

Step 4: Zapier Logic to Burn the Token

- When a QR is scanned:
 - Zapier updates the “Status” to **Burned**
 - Records the timestamp
 - (Optional) notifies the original buyer that ticket was used
-

Step 5: Logging and Audit Trail

- Use another tab in the Sheet for:
 - All burns
 - Timestamps
 - Device/location info (if available)
 - Rejected scans (duplicate attempts)
-

What You’ll End Up With:

- A working **QR-based ticketing prototype**

- Validated at the door with a **burn-on-scan mechanism**
 - A mobile-accessible scanner that **requires no coding**
 - Full audit trail of usage
-

Ready to build it live?

I can help you:

1. Set up the exact Google Sheet structure
2. Configure Glide and Zapier step-by-step
3. Automate the QR-to-burn process

Which step should we do first? Or do you want the full template zipped and sent to get rolling?